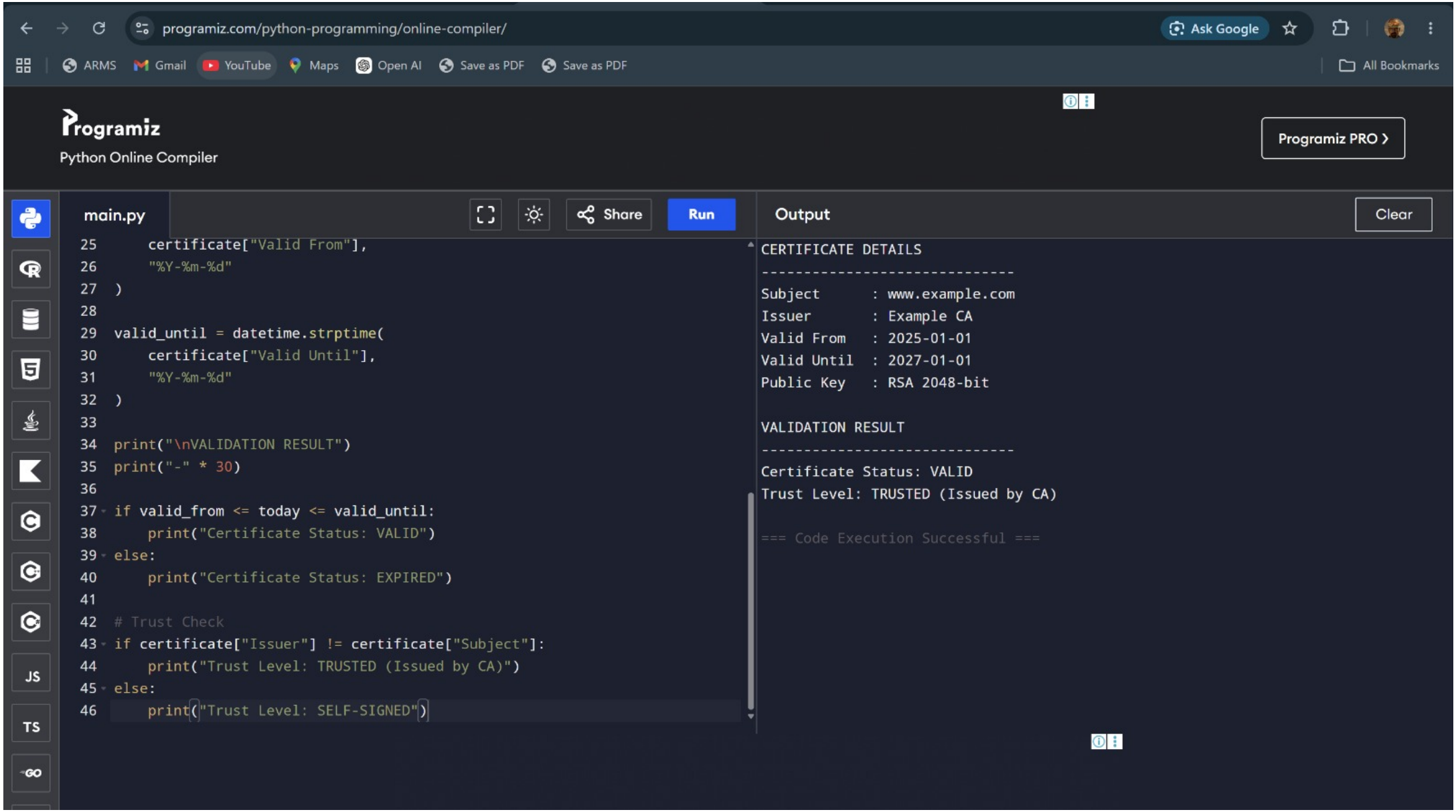

programiz.com/python-programming/online-compiler/
Ask Google
ARMS
Gmail
YouTube
Maps
Open AI
Save as PDF
Save as PDF
All Bookmarks
Programiz
Python Online Compiler
Programiz PRO ›
main.py
Share
Run
Output
Clear
25 certificate["Valid From"],
26 "%Y-%m-%d"
27)
28
29 valid_until = datetime.strptime(
30 certificate["Valid Until"],
31 "%Y-%m-%d"
32)
33
34 print("\nVALIDATION RESULT")
35 print("-" * 30)
36
37 if valid_from <= today <= valid_until:
38 print("Certificate Status: VALID")
39 else:
40 print("Certificate Status: EXPIRED")
41
42 # Trust Check
43 if certificate["Issuer"] != certificate["Subject"]:
44 print("Trust Level: TRUSTED (Issued by CA)")
45 else:
46 print("Trust Level: SELF-SIGNED")
CERTIFICATE DETAILS
Subject : www.example.com
Issuer : Example CA
Valid From : 2025-01-01
Valid Until : 2027-01-01
Public Key : RSA 2048-bit
VALIDATION RESULT
Certificate Status: VALID
Trust Level: TRUSTED (Issued by CA)
=== Code Execution Successful ===

programiz.com/python-programming/online-compiler/

ARMS | Gmail | YouTube | Maps | Open AI | Save as PDF | Save as PDF | All Bookmarks

Programiz
Python Online Compiler

Programiz PRO >

main.py | Share | Run

```
5
6       with open(filename, "rb") as file:
7           while chunk := file.read(4096):
8               sha256.update(chunk)
9
10      return sha256.hexdigest()
11
12  filename = input("Enter file name: ")
13
14  original_hash = calculate_sha256(filename)
15
16  print("\nSHA-256 Hash:")
17  print(original_hash)
18
19  print("\nModify the file and save it.")
20  input("Press Enter after modification...")
21
22  new_hash = calculate_sha256(filename)
23
24  print("\nNew SHA-256 Hash:")
25  print(new_hash)
26
27  if original_hash == new_hash:
28      print("\nFile Integrity Verified.")
29  else:
30      print("\nTampering Detected!")
```

Output | Clear

```
Enter file name: Hello World

This is a file integrity verification experiment.
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 14, in <module>
  File "<main.py>", line 6, in calculate_sha256
FileNotFoundError: [Errno 2] No such file or directory: 'Hello World'

=== Code Exited With Errors ===
```

programiz.com/python-programming/online-compiler/

Ask Google | ARMS | Gmail | YouTube | Maps | Open AI | Save as PDF | Save as PDF | All Bookmarks

Programiz
Python Online Compiler

Programiz PRO >

main.py | Share | Run

```python
import hashlib
import time

# Sample dataset
dataset = [
    "Hello World",
    "Cyber Security",
    "Hash Functions",
    "MD5 Algorithm",
    "SHA-256 Algorithm",
    "Data Integrity",
    "Network Security"
]

print("=" * 80)
print("MD5 vs SHA-256 Hash Comparison")
print("=" * 80)

# MD5 Hashing
start_md5 = time.perf_counter()

md5_hashes = []
for data in dataset:
    hash_value = hashlib.md5(data.encode()).hexdigest()
    md5_hashes.append(hash_value)

```

Output | Clear

```
================================================================================
    =
MD5 vs SHA-256 Hash Comparison
================================================================================
    =

Hash Values:

Data      : Hello World
MD5       : b10a8db164e0754105b7a99be72e3fe5
SHA-256   : a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
--------------------------------------------------------------------------------
    -
Data      : Cyber Security
MD5       : 4690cd84196b0a4e3f84f2cc3bf79ac7
SHA-256   : 805e20cf7c3f85210a1b22b68895e2f7edff2cf1565bb685b77f87a3126db27e
--------------------------------------------------------------------------------
    -
Data      : Hash Functions
MD5       : 354623b3c73aef49017cc5d8b15d85c3
SHA-256   : c4cad6854759393c8c1fee1107af989ccb66443362dda102a199dc73dff70be5
--------------------------------------------------------------------------------
    -
Data      : MD5 Algorithm
MD5       : 59f63fd230551d3fb954e8e61819e7a3
SHA-256   : 2c711462a9c78f809cdcb35f9c79b717040ffdf46ba480b9270c50659979f800
```

Hide code

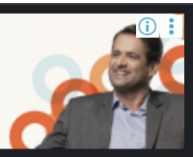
```python
from cryptography.hazmat.primitives.asymmetric import
from cryptography.hazmat.primitives import hashes

# Generate RSA Keys
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

message = b"Secure Message"

# Sign Message
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("Signature Generated")

# Verify Signature
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
```

Console

Run

```
Run started
Initializing environment
Installing packages
Running code
Signature Generated
Signature Verified
Run completed in 7979.199999988079ms
```

Programiz
Python Online Compiler



Programiz PRO ›

main.py

Share Run

```
10      }
11
12      return ticket
13
14  # Service Validation
15  def validate_ticket(ticket):
16      current_time = int(time.time())
17
18      if current_time - ticket["timestamp"] < 30:
19          return True
20      else:
21          return False
22
23  user = input("Enter username: ")
24
25  ticket = generate_ticket(user)
26
27  print("\nTicket Issued:")
28  print(ticket)
29
30  print("\nValidating Ticket...")
31
32  if validate_ticket(ticket):
33      print("Access Granted")
34  else:
35      print("Access Denied")
```

Output Clear

```
Enter username: nikith

Ticket Issued:
{'user': 'nikith', 'timestamp': 1781501118}

Validating Ticket...
Access Granted

=== Code Execution Successful ===
```